Apptracker v7.3 – Project Case Study

A Modular Python Automation and Reporting System

Author: Thomas Barreto
Status: Production Release (v7.3)

Website: https://thomasbarreto.com/

Executive Summary

Apptracker was created to solve a real world problem: managing and tracking job applications manually. What began as a simple Python utility for counting PDF application records evolved through multiple releases into a production ready automation and reporting system for macOS.

Version 7.0 focused on improving the application architecture without changing the established production workflow. The application was refactored into focused Python modules while retaining `apptracker.py` as the production entry point and workflow orchestrator.

The v7.0 release also corrected an exception and audit interaction issue, improved reliability when application folders are empty, and was verified against real production data with matching results between the final v6.0 workflow and the v7.0 release. Version 7.1 followed with a targeted fix that made PDF creation-date retrieval cross-platform rather than macOS only. Version 7.2 completed the Windows-compatibility pass by removing macOS-specific wording and non-ASCII characters from the email notification module. Version 7.3 added filename validation logging, so PDFs that do not match the expected naming pattern are now flagged in the console and in Audit.csv, and improved the Validation Dashboard column layout for readability.

Business Problem

Maintaining accurate application records, reporting results, and execution history manually was repetitive and error prone. The process required a reliable system that could automate data collection, generate consistent reports, validate results, maintain an audit trail, and operate with minimal user intervention.

Project Objectives

- Automate job application tracking
- Generate Excel and CSV reports
- Validate generated output automatically
- Maintain an audit trail
- Support optional email notifications
- Generate application and validation dashboards
- Support unattended scheduled execution
- Improve application maintainability through modular architecture
- Preserve the existing production workflow during the v7.0 refactor

Solution Overview

Apptracker reads configuration settings, validates application folders, scans Active and Rejected PDF records, extracts application metadata, generates Excel and CSV reports, validates the output, updates dashboards, records audit activity, optionally sends email notifications, and can execute automatically through macOS `launchd`.

Version 7.0 separates these responsibilities into focused modules while preserving the established production interface and workflow.

`apptracker.py` remains the production entry point and coordinates the application workflow.

v7.0 Modular Architecture

The application was refactored into focused modules with clear responsibilities:

- `apptracker.py` – Production entry point and application workflow orchestration
- `config.py` – Configuration loading and folder validation
- `collector.py` – PDF scanning, metadata extraction, and application record collection
- `report.py` – Excel report generation and CSV exports
- `validation.py` – PDF, Excel, and CSV validation and validation history
- `audit.py` – Audit logging, runtime tracking, run summaries, and exception handling
- `notify.py` – User Report and Admin Summary email notifications
- `dashboards.py` – Application and validation dashboard generation
- `demo.py` – Optional demo generation integration

- `xlsx_utils.py` – Shared Excel utilities
- `make_demo_data.py` – Demo data and workbook generation

This architecture improves readability, maintainability, troubleshooting, reuse, and separation of responsibilities while preserving the existing application behavior.

Application Workflow

```
Settings.txt
      |
      v
Configuration and Folder Validation
      |
      v
PDF Collection and Metadata Extraction
      |
      v
Excel and CSV Reporting
      |
      v
Application Dashboard Generation
      |
      v
PDF, Excel, and CSV Validation
      |
      v
Validation Dashboard Generation
      |
      v
Audit Logging
      |
      v
Optional Email Notifications
      |
      v
Optional Demo Generation
      |
      v
Application Complete
```

Technology Stack

- Python
- pandas
- openpyxl
- keyring
- Excel
- CSV
- macOS Automator
- macOS `launchd`
- macOS Keychain
- SMTP

Key Features

- Configuration driven application design
- PDF application collection
- Application metadata extraction
- Excel reporting
- CSV exports
- Application dashboard generation
- Validation dashboard generation
- Automated PDF, Excel, and CSV cross checking
- Audit logging and execution history
- User Report email notifications
- Admin Summary email notifications
- Optional demo data generation
- macOS application launcher
- Manual command launcher
- Unattended scheduled execution

Development Journey

The project was developed through iterative releases, with each version adding functionality or improving the application architecture.

Major milestones included:

Version	Milestone
v1–v2	PDF application counter
v3	CSV export
v4	Application refactor
v5.1	Excel reporting
v5.2	Configuration and validation
v5.3	Application dashboard
v5.4	Metadata and timestamp enhancements
v5.5	Validation dashboard
v5.6	Demo data automation
v5.7	Audit logging
v5.8	Email automation
v5.9	macOS application launcher
v6.0	Scheduled automation using `launchd`
v7.0	Modular architecture and reliability improvements
v7.1	Cross-platform PDF creation-date retrieval
v7.2	Windows-compatible notification wording and output
v7.3	Filename validation logging and dashboard layout fix

v7.0 Challenges and Solutions

Modular Refactor While Preserving Production Behavior

The primary v7.0 challenge was improving the internal architecture without changing the established application workflow, outputs, launchers, or scheduler integration.

The solution was to retain `apptracker.py` as the production entry point and move focused responsibilities into dedicated modules. This preserved compatibility with the existing macOS application launcher, manual

command launcher, and `launchd` scheduler while making the code easier to maintain and troubleshoot.

Exception and Audit Interaction

The application exception boundary and the `Audit` uncaught exception handler required correction to ensure unexpected failures were recorded properly.

The updated design records the `FAILED` audit state before re raising the original exception. The exception then reaches the `Audit` exception handler with the original traceback and expected failure exit behavior preserved.

Empty Folder Reliability

The application previously encountered a division by zero error when both the Active and Rejected folders contained no PDF records.

Zero total safeguards were added to the percentage calculations. Apptracker can now complete reporting, validation, audit logging, and dashboard generation successfully when both application folders are empty.

Verification

The v7.0 modular refactor was verified against real production data.

Testing confirmed:

- Matching results between the final v6.0 workflow and the v7.0 production workflow
- Normal report generation
- Successful Excel and CSV output
- Successful validation processing
- Normal dashboard generation
- Correct audit logging
- Normal email notification processing
- Successful demo generation
- Correct failure recording during unexpected exceptions
- Successful completion when both application folders are empty

The verification confirmed that the v7.0 release preserved the established production behavior while improving application organization and reliability.

Final Outcome

Apptracker evolved from a simple PDF counting utility into a modular Python automation and reporting system capable of collecting application data, generating reports and dashboards, validating output, maintaining audit history, sending notifications, generating demo data, and executing automatically on a schedule.

Version 7.0 improved the internal architecture through focused modules while retaining `apptracker.py` as the production entry point. The release preserved the existing production workflow, launchers, scheduler integration, inputs, and outputs while adding defensive reliability improvements.

Lessons Learned

This project strengthened skills in:

- Python application architecture
- Modular software design
- Workflow orchestration
- Data collection and transformation
- Excel and CSV reporting
- Automated validation
- Dashboard development
- Audit logging
- Exception handling
- macOS automation
- Scheduled execution
- Debugging and reliability testing
- Iterative software delivery

The project also demonstrated the value of building software incrementally, preserving stable production behavior during architectural changes, and validating refactored applications against real production data.